Algorithmen – eine kurze Einführung

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Hilfsklasse Arrays
- Unterstützung durch IDE
- Visualisierung von Arrays in der IDE

Was wird in diesem Foliensatz besprochen?

- Algorithmen allgemein
- Motivierendes Beispiel

Was lernen Sie kennen?

- Was Algorithmen ausmacht
- Unterschiedliche Herangehensweisen für Problemlösen

Bisherige Beispiele für Algorithmen in dieser Vorlesung

Euklidischer Algorithmus (ggT-Berechnung)

Zahlensummen bilden

Primzahlensuche

Matrizenmultiplikation

Definition

*Ein Algorithmus ist ein **endliches**, **schrittweises** Verfahren zur **Berechnung** von gesuchten aus gegebenen Größen, in dem jeder **Schritt** aus einer Anzahl **ausführbarer**, **eindeutiger** Operationen und einer **Angabe über den nächsten Schritt** besteht.*

[Peter Rechenberg, *Was ist Informatik?: Eine allgemeinverständliche Einführung*, Hanser Fachbuch, 3 Auflage, 2000]

Endlichkeit

Statische Endlichkeit

- Algorithmus muss mit **endlich vielen Zeichen** beschreibbar sein

Dynamische Endlichkeit

- Ein Algorithmus darf zu jedem Zeitpunkt während seiner Ausführung nur **endlich viel Speicherplatz** benötigen

Algorithmus und Berechnung

Ein Algorithmus transformiert Eingabedaten in Ausgabedaten („berechnet“ Ausgabedaten)

Der Begriff **Berechnung** ist dabei allgemeiner zu verstehen, z.B.

- Berechnungen im klassischen Sinn (Addition, Subtraktion etc.)
- Manipulation von Texten
- Verarbeitung von beliebigen Signalen (z. B. Bilder) oder Sequenzen (von Zahlen etc.)

Annahme für diese Vorlesung

- Die Eingabedaten stehen schon zu Beginn fest

Eindeutigkeit und Ausführbarkeit

Eindeutigkeit	Ausführbarkeit
• Algorithmus besteht aus einer Folge von **eindeutigen Schritten** • Computer „denkt“ nicht mit, sondern führt die Befehle der Reihe nach aus	• Jeder Schritt muss **durch den Computer ausführbar** sein

Determiniertheit und Determinismus

Determiniertheit
• Ein Algorithmus liefert bei **gleichen Voraussetzungen** (Parameter, Startbedingungen) stets das **gleiche Ergebnis**

Determinismus
• Zu **jedem Zeitpunkt** der Ausführung besteht **höchstens eine Möglichkeit** der Fortsetzung • Der gesamte Ablauf ist eindeutig bestimmt

Es gilt
• Jeder deterministische Algorithmus ist determiniert • Nicht jeder determinierte Algorithmus ist deterministisch

Terminierung

Algorithmus muss für alle möglichen Eingaben nach **endlich vielen Verarbeitungsschritten** anhalten und ein Resultat liefern

Ob ein (beliebiger) Algorithmus mit einer beliebigen Eingabe terminiert, ist nicht durch einen Algorithmus lösbar (Halteproblem)

Algorithmen und Problemlösen

Ein Algorithmus löst immer ein allgemeines Problem

(eine Klasse von Problemen)

- Problem beschreibt, was gelöst wird
- Algorithmus beschreibt, wie es gelöst wird

Algorithmen machen oft nur einen kleinen Teil eines Programms aus, sie bestimmen aber oft, wie effizient dieses Programm ist

Beispiel – Maximale Abschnittssumme

Finden der maximalen Abschnittssumme in einem Array von *n* Zahlen (positive und negative Zahlen)

- Ist die Teilfolge von aufeinanderfolgenden Zahlen, die unter allen möglichen Teilfolgen die größte Summe liefert

-59	**52**	**46**	**14**	**-50**	**58**	**-87**	**-77**	**34**	**15**

Maximale Summe = 52 + 46 + 14 + -50 + 58 = 120

Maximale Abschnittssumme – Spezialfälle

Wenn alle Werte > 0 sind, dann entspricht die maximale Summe der Summe aller Elemente

Wenn alle Werte < 0 sind, dann entspricht die maximale Summe dem Element mit dem kleinsten Absolutbetrag

Ein leeres Array ist nicht zulässig

Es kann mehrere Teilfolgen geben, die die maximale Summe ergeben es wird aber nur eine Summe ermittelt und zurückgeliefert

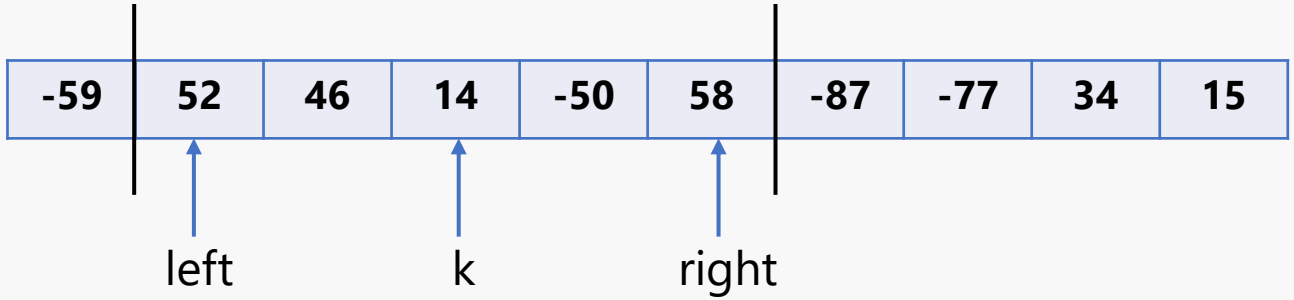
1. Variante – Idee

Idee

- Alle Teilfolgen bilden, die im Array enthalten sind
- Die Werte einer Teilfolge aufaddieren
- Die maximale von all diesen Summen bestimmen

Benötigt werden

- Linker und rechter Rand der Teilfolge (left, right) und aktuelle Position in der Teilfolge (k)



1. Variante – Implementierung

```java
private static int maxSum1(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        for (int right = left; right < input.length; right++) {
            int sum = 0;
            for (int k = left; k <= right; k++) {
                sum += input[k];
            }
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Beispiel für Ablauf (mit Ausgabe)

```java
private static int maxSum1(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        for (int right = left; right < input.length; right++) {
            int sum = 0;
            for (int k = left; k <= right; k++) {
                sum += input[k];
                System.out.println("left = " + left + ", right = " + right + ", k = " + k + ", value = " + input[k] + ", sum = " + sum);
            }
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Erzeugt durch folgenden Aufruf:
maxSum1(**new int**[]{2, 3, -6, 4})

```
left = 0, right = 0, k = 0, value = 2, sum = 2
left = 0, right = 1, k = 0, value = 2, sum = 2
left = 0, right = 1, k = 1, value = 3, sum = 5
left = 0, right = 2, k = 0, value = 2, sum = 2
left = 0, right = 2, k = 1, value = 3, sum = 5
left = 0, right = 2, k = 2, value = -6, sum = -1
left = 0, right = 3, k = 0, value = 2, sum = 2
left = 0, right = 3, k = 1, value = 3, sum = 5
left = 0, right = 3, k = 2, value = -6, sum = -1
left = 0, right = 3, k = 3, value = 4, sum = 3
left = 1, right = 1, k = 1, value = 3, sum = 3
left = 1, right = 2, k = 1, value = 3, sum = 3
left = 1, right = 2, k = 2, value = -6, sum = -3
left = 1, right = 3, k = 1, value = 3, sum = 3
left = 1, right = 3, k = 2, value = -6, sum = -3
left = 1, right = 3, k = 3, value = 4, sum = 1
left = 2, right = 2, k = 2, value = -6, sum = -6
left = 2, right = 3, k = 2, value = -6, sum = -6
left = 2, right = 3, k = 3, value = 4, sum = -2
left = 3, right = 3, k = 3, value = 4, sum = 4
```

2. Variante – Idee

1. Variante ist ineffizient

- Innerste Schleife (Summation der Folgenelemente) ist nicht notwendig

Beispiel für Problem

- *left*=2 und *right*=4
  - Die innerste Schleife addiert die Elemente mit Indizes 2, 3 und 4
- Danach folgt *left*=2 und *right*=5
  - Die innerste Schleife addiert die Elemente mit Indizes 2, 3, 4 und 5
  - **In diesem Fall müsste nur das Element am Index 5 zur vorherigen Summe addiert werden**

2. Variante – Implementierung

```java
private static int maxSum2(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        int sum = 0;
        for (int right = left; right < input.length; right++) {
            sum += input[right];
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Beispiel für Ablauf (mit Ausgabe)

```java
private static int maxSum2(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        int sum = 0;
        for (int right = left; right < input.length; right++) {
            sum += input[right];
            System.out.println("left = " + left + ", right = " + right + ", value: " + input[right] + ", sum " + sum);
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Erzeugt durch folgenden Aufruf:
maxSum2(**new int**[]{2, 3, -6, 4})

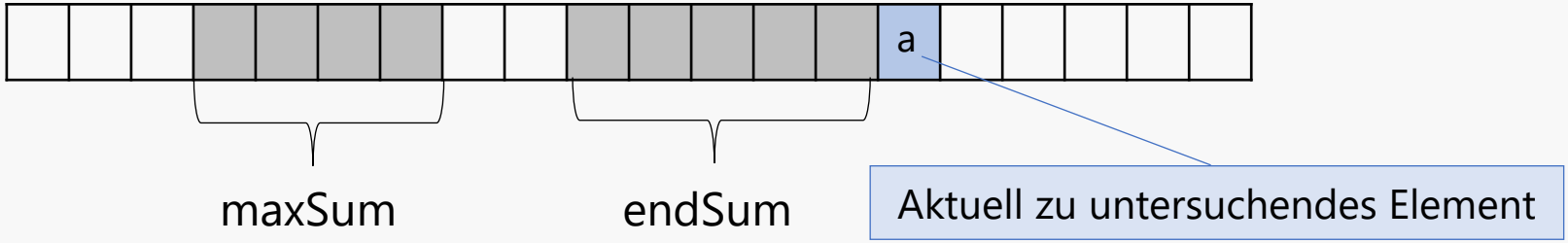
```
left = 0, right = 0, value: 2, sum 2
left = 0, right = 1, value: 3, sum 5
left = 0, right = 2, value: -6, sum -1
left = 0, right = 3, value: 4, sum 3
left = 1, right = 1, value: 3, sum 3
left = 1, right = 2, value: -6, sum -3
left = 1, right = 3, value: 4, sum 1
left = 2, right = 2, value: -6, sum -6
left = 2, right = 3, value: 4, sum -2
left = 3, right = 3, value: 4, sum 4
```

3. Variante – Idee (1)

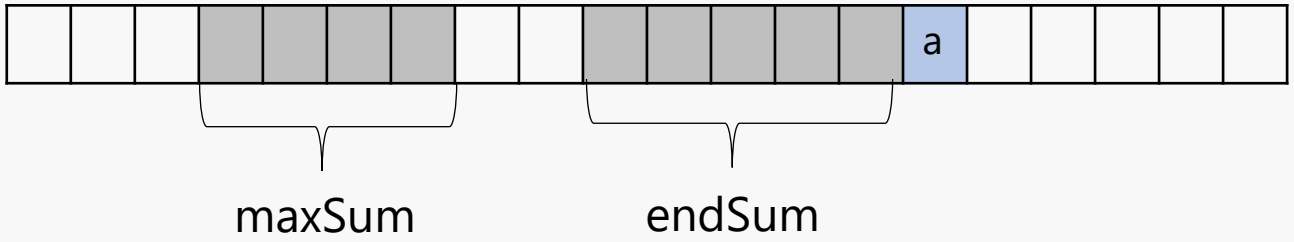
Array wird von links nach rechts einmal durchlaufen

Informationen an der aktuellen Stelle

- Die maximale Summe einer Teilfolge im gesamten bisher inspizierten Anfangsstück (maxSum)
- Die an der aktuellen Stelle endende Summe der aktuellen Teilfolge (endSum)



3. Variante – Idee (2)



Maximale Teilsumme
• endSum′ ist das Maximum von endSum + a und a • maxSum′ ist das Maximum von maxSum und endSum′

3. Variante – Implementierung

```
private static int maxSum3(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int pos = 1; pos < input.length; pos++) {
        endSum = Math.max(endSum + input[pos], input[pos]);
        maxSum = Math.max(maxSum, endSum);
    }
    return maxSum;
}
```

Beispiel für Ablauf (mit Ausgabe)

```java
private static int maxSum3(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int pos = 1; pos < input.length; pos++) {
        endSum = Math.max(endSum + input[pos], input[pos]);
        System.out.println("pos = " + pos + ", value " + input[pos] + ", sum " + endSum);
        maxSum = Math.max(maxSum, endSum);
    }
    return maxSum;
}
```

Erzeugt durch folgenden Aufruf:
maxSum3(**new int**[]{2, 3, -6, 4})

```
pos = 1, value 3, sum 5
pos = 2, value -6, sum -1
pos = 3, value 4, sum 4
```

Maximale Abschnittssumme – Vergleich

Unterschied zwischen den drei Versionen?

- Ergebnis: Kein Unterschied
- Laufzeit für große Arrays: **Enorme Unterschiede**